Sleep-Time Consolidation for Continual RLVR

Idea Sketch — Backburner

Rohan Sikand · Stanford University

March 2026

The Neuroscience Analogy

During waking hours, the brain learns new experiences (hippocampal encoding). During sleep, it replays and consolidates these experiences, integrating them with prior knowledge (cortical consolidation).

Biological sleep:

- Sharp-wave ripples replay recent experiences
- Interleaved with older memories
- Transfer from hippocampus to cortex
- Strengthens important connections
- Prunes irrelevant ones

RLVR analog:

- Model self-generates across all tasks
- Mixes new task solutions with old
- Verifier filters for quality
- Diversity loss maintains breadth
- Low learning rate, no new environment interaction

Between RLVR stages, add a "sleep" phase that consolidates without new data.

The Problem Sleep Solves

Sequential RLVR: $\xrightarrow{\text{RLVR (math)}} \xrightarrow{\text{RLVR (code)}} \xrightarrow{\text{RLVR (science)}}$

Each transition is abrupt. The model goes from `maximize math reward''` tomaximize code reward" with no transition period. The distribution shifts sharply. Math representations that were being reinforced are suddenly abandoned.

Sleep-time consolidation:

$$\xrightarrow{\text{RLVR (math)}} \xrightarrow{\textbf{sleep}} \xrightarrow{\text{RLVR (code)}} \xrightarrow{\textbf{sleep}} \xrightarrow{\text{RLVR (science)}}$$

The sleep phase has no task-specific objective. Instead, the model:

1. Self-generates solutions across *all* prior tasks
2. Filters correct solutions via verifiers

The Sleep Phase Algorithm

Input: Model π_θ after RLVR stage j. Problem sets $\mathcal{S}_1, \ldots, \mathcal{S}_j$ from all prior tasks. Verifiers $V_1, \ldots, V_j$.

For each sleep step:

1. **Multi-task self-generation:** For each task $k \in \{1, \ldots, j\}$:
   - Sample problems from $\mathcal{S}_k$
   - Generate N rollouts per problem from π_θ
   - Verify with V_k, keep correct rollouts
2. **Diversity filtering:** Among correct rollouts for each problem, select a diverse subset (maximize embedding distance within the set)
3. **Consolidation update:**

$$\theta \leftarrow \theta - \eta_{\text{sleep}} \nabla_\theta \left[\mathcal{L}_{\text{SFT}}^{(\text{diverse correct})} + \alpha \, \mathcal{L}_{\text{diversity}} + \beta \, \mathcal{L}_{\text{pred}}^{(\text{all tasks})} \right]$$

What Each Loss Does During Sleep

$\mathcal{L}_{\text{SFT}}^{(\text{diverse correct})}$: Reinforce correct solutions across all tasks

Standard SFT on self-generated, verifier-filtered solutions from all prior tasks. Keeps the model's generation capability alive across domains. Uses diverse subset to prevent mode collapse within each task.

$\mathcal{L}_{\text{diversity}}$: Prevent consolidation from collapsing to one mode

Encourage the model to maintain multiple solution strategies per task. Could be embedding-space diversity, token-level entropy on correct solutions, or forward-KL from the pre-sleep checkpoint.

$\mathcal{L}_{\text{pred}}^{(\text{all tasks})}$: Maintain internal execution models

Feedback prediction across all task domains. Keeps the comprehension side alive alongside the generation side (from the SFT loss). Same mechanism as the continual

Why Sleep, Not Concurrent Rehearsal?

We already proposed concurrent methods ($\mathcal{L}_{\text{pred}}$ during code RLVR, self-gen replay during code RLVR). Why add a separate sleep phase?

Gradient interference. During code RLVR, the primary gradient points toward code optimization. The rehearsal gradient (math maintenance) points elsewhere. These compete. The code gradient is large (primary objective); the rehearsal gradient is small (secondary, tempered by λ). In practice, the primary objective dominates and rehearsal provides only partial protection.

During sleep, there is no competing primary objective. All gradient signal is consolidation. Math, code, and science representations are reinforced equally. No task dominates. The model integrates capabilities rather than trading them off.

Analogy: Studying for exam A while in exam B's class is hard — your attention is split.

The Consolidation Hypothesis

Claim: Sleep doesn't just prevent forgetting — it produces **better representations** than either sequential RLVR or concurrent rehearsal alone.

Why? During RLVR on task A, representations are shaped for task A's reward landscape. During RLVR on task B, they're reshaped for task B's. The representations are always task-specialized, never task-general.

During sleep, the model trains on all tasks simultaneously with no single task dominating. The optimizer finds representations that serve **all tasks well** — shared reasoning circuits that are reinforced by every task rather than overwritten by each new one.

Testable prediction: After sleep consolidation, the model should perform better on *cross-domain* problems (math + code hybrid questions) than either sequential RLVR or concurrent rehearsal, because the sleep phase builds shared representations that bridge

Connection to Sleep-Time Compute

Sleep-time consolidation naturally connects to the broader "sleep-time compute" research direction:

Phase	Standard RLVR	With sleep
Task A training	Active RLVR	Active RLVR
Between tasks	Idle / wasted compute	**Sleep consolidation**
Task B training	Active RLVR	Active RLVR
Between tasks	Idle / wasted compute	**Sleep consolidation**

The time between RLVR stages is usually idle (model evaluation, data preparation for next stage). Sleep consolidation uses this idle compute productively. You're not adding wall-clock time — you're using time that was already spent waiting.

This framing — "use inter-stage compute for consolidation rather than letting it idle" —

Consolidation Schedule

How long should the model sleep?

Schedule	Description	Trade-off
Fixed	S sleep steps after every RLVR stage	Simple; may over/under-consolidate
Adaptive	Sleep until cross-task eval stabilizes	Efficient; needs eval metric
Proportional	Sleep steps $= \gamma\times$ RLVR steps	Scales with training duration
Convergence	Sleep until self-gen diversity plateaus	Principled; compute-variable

Adaptive is recommended: monitor a held-out eval set spanning all prior tasks. Sleep until performance on all tasks is within some tolerance of their post-RLVR peaks. This automatically adjusts: if forgetting was mild, sleep is short. If forgetting was severe, sleep is longer.

Learning rate during sleep: $\eta_{\text{sleep}} = 0.1 \times \eta_{\text{RLVR}}$. Gentle updates that consolidate without disrupting

What Sleep Looks Like in Practice

Example: Training a model on math, code, and science.

Stage 1: RLVR on math for 200 steps. Math Pass@1 = 65%.

Sleep 1: Consolidation for 50 steps. Self-generate math solutions, verify, SFT with diversity. Math Pass@1 maintained at 64%.

Stage 2: RLVR on code for 200 steps. Code Pass@1 = 58%. Math Pass@1 drops to 48%.

Sleep 2: Consolidation for 80 steps on math + code. Self-generate both, verify, consolidate. Math recovers to 60%. Code maintained at 57%.

Stage 3: RLVR on science for 200 steps. Science Pass@1 = 52%. Math drops to 55%. Code drops to 50%.

Sleep 3: Consolidation for 100 steps on all three. Math recovers to 58%, code to 55%,

Comparison to Other Continual Methods

Method	When?	Mechanism	Prevents?	Recovers?
Concurrent $\mathcal{L}_{\text{pred}}$	During RLVR	Aux loss	Partially	No
Self-gen replay	During RLVR	SFT on self-gen	Partially	No
Teacher reg	During RLVR	Logit matching	Partially	No
GLM-5 recovery	After all stages	Distillation	No	Yes
Sleep consolidation	**Between stages**	**Self-gen + div.**	**Partially**	**Yes**

Sleep is unique in being both **preventive and restorative**. Concurrent methods only prevent (they slow forgetting during training). GLM-5 only restores (it repairs after the damage). Sleep does both: it recovers what was lost in the previous stage AND strengthens representations against future forgetting.

Combining Sleep with Concurrent Methods

The methods are complementary:

$$\xrightarrow[+\,\mathcal{L}_{\text{pred}}^{(\text{math})}]{\text{RLVR (code)}} \xrightarrow{\textbf{sleep (math + code)}} \xrightarrow[+\,\mathcal{L}_{\text{pred}}^{(\text{math, code})}]{\text{RLVR (science)}} \xrightarrow{\textbf{sleep (all)}}$$

During RLVR: $\mathcal{L}_{\text{pred}}$ slows forgetting (Tier 2 intervention).

Between stages: Sleep recovers what was lost and consolidates.

Prediction: The combination outperforms either alone. Concurrent rehearsal limits the damage; sleep repairs the residual damage and builds cross-task representations. Without concurrent rehearsal, sleep has more to recover. Without sleep, concurrent rehearsal accumulates residual degradation.

Experimental Plan

Phase 1: Baseline + diagnostic (1 week)

- Sequential RLVR: math (200) then code (200) then science (200)
- Measure all-task Pass@1 at every checkpoint
- Establish forgetting curve without any intervention

Phase 2: Sleep phase design (2 weeks)

- Insert 50-step sleep phases between each stage
- Ablate: SFT-only sleep vs. SFT + diversity vs. SFT + diversity + $\mathcal{L}_{\text{pred}}$
- Measure recovery magnitude and consolidation quality

Phase 3: Comparison (3 weeks)

- (a) No intervention, (b) concurrent $\mathcal{L}_{\text{pred}}$, (c) self-gen replay, (d) sleep only, (e) concurrent + sleep, (f) GLM-5 recovery

Risks

Risk	Severity	Mitigation
Sleep is just SFT on self-gen data (not novel enough)	**High**	The novelty is the schedule + diversity + consolidation framing; needs strong empirical story
Self-gen solutions are too low quality after forgetting	High	Combine with $\mathcal{L}_{\text{pred}}$; use high sampling budget
Sleep adds wall-clock time	Medium	Use idle inter-stage compute; show it replaces GLM-5's recovery stage
Diversity loss doesn't prevent mode collapse during sleep	Medium	Ablate diversity mechanisms
Cross-domain emergence	Medium	It's a bonus finding, not

Honest Assessment

Pros:

- Intuitive neuroscience-inspired framing (sleep as consolidation)
- Both preventive and restorative (unique among methods)
- Uses idle inter-stage compute productively
- Consolidation hypothesis is testable and could yield surprise findings (cross-domain emergence)
- Complementary with concurrent methods — natural combination
- No labels, no stored data (same self-generating property)

Cons:

- "Sleep" framing might feel gimmicky — substance is SFT on diverse self-gen data